1. What is C programming language and explain about features of C?

What is C Programming Language?

C programming language is a general-purpose, procedural oriented programming language developed in 1972 by Dennis Ritchie at Bell Labs.
It is mainly used for system programming, such as developing operating systems, compilers, and embedded systems.

C became very popular because it provides low-level memory access and high-level programming features, making it powerful and efficient.

For example, the operating system Unix was largely written in C.

Features of C Programming Language

1. Simple and Efficient

C has a **simple syntax** and requires fewer keywords. This makes it easier to learn and efficient for writing programs.

2. Structured Programming Language

C supports **structured programming**, meaning programs can be divided into smaller parts using **functions**, which improves readability and maintenance.

3. Portable (Machine Independent)

Programs written in C can run on **different computers with little or no modification**, making it portable.

4. Rich Library Functions

C provides many **built-in library functions** (like `printf()`, `scanf()`, `strlen()`, etc.) that make programming easier.

5. Low-Level Memory Access

C allows direct **memory manipulation using pointers**, which is useful in system programming and embedded systems.

6. Fast Execution

Programs written in C are **very fast** because they are compiled directly into machine code.

7. Extensible

Users can **add their own functions** and libraries, making C highly extensible.

8. Middle-Level Language

C is called a **middle-level language** because it supports both:

- **High-level programming** (like functions and loops)
- **Low-level programming** (like pointers and memory access)

(OR)

- C Programming is a Procedure oriented programming language .It was developed denis rich in 1972.
- C is very simple and important features and it provides foundation for many other languages c++,java,python.
- Mainly C is developed for embedded system,os,device drivers,micro controller,complier and interpreter.
- C is a simple and fast programming language.
- It can allow low level memory access by using pointers
- C is also used to embedded system applications,network applications,desktop applications,database application and general purpose and also gaming applications.

2. Explain about advantages and disadvantages of C?

Advantages of C

1. **High Performance**
    - C is a compiled language, producing efficient machine code, making it suitable for performance-critical applications such as operating systems, embedded systems, and game engines.
2. **Low-Level Access**
    - C provides direct access to memory using pointers, enabling fine control of hardware resources. This feature is particularly useful in system programming.
3. **Portability**
    - Code written in C can be compiled on multiple platforms with minimal modifications, making C a portable language.
4. **Modularity**

- Functions in C help in breaking large programs into smaller, manageable modules improving readability and maintainability.

5. **Foundation of Other Languages**
   - Many modern languages like C++, Java, and Python derive features or syntax from C. Learning C provides a strong programming foundation.
6. **Rich Library Function Support**
   - C offers a set of standard libraries for operations such as input/output handling, string manipulation, and mathematics, reducing development effort.
7. **Extensibility**
   - It allows you to create your functions and extend language features according to program requirements.

Disadvantages of C

1. **Lack of Object-Oriented Features**
   - C is not object-oriented and does not support classes, inheritance, or polymorphism, limiting its suitability for large-scale, object-oriented software projects.
2. **Manual Memory Management**
   - Developers must manually allocate and free memory using `malloc` and `free`, increasing the risk of memory leaks and pointer-related bugs.
3. **No Built-in Exception Handling**
   - Error handling in C requires explicit checks and code, unlike languages like Java or Python that provide structured exception handling.
4. **Limited Standard Library Support**
   - Compared to higher-level languages, C does not provide advanced libraries for graphics, GUI development, or networking out-of-the-box.
5. **Complex Syntax for Beginners**
   - Features like pointers, memory management, and bit-level operations can be challenging for new programmers to master.
6. **No Bounds Checking**
   - Arrays and pointer operations are unchecked, which can lead to undefined behavior and security vulnerabilities.

3. Explain various applications of C?

System Programming

- C is extensively used in system-level programming such as operating systems, device drivers, and embedded systems.

- **Example**: Unix and Linux kernel development predominantly uses C because it offers low-level access to memory and hardware, enabling efficient system calls and resource management.

2. Embedded Systems

- C is ideal for programming microcontrollers, IoT devices, and industrial automation systems. Its ability to directly manipulate hardware registers and memory makes it a preferred choice for resource-constrained environments.
- **Example**: Programs for Arduino, PIC, and ARM microcontrollers are often written in C.

3. Software Development

- C is the foundation for many high-level languages and compilers, making it suitable for building compilers, interpreters, and other software development tools.
- **Example**: GCC (GNU Compiler Collection) is implemented in C.

4. Game Development

- Many game engines use C for performance-critical components because it allows direct access to memory and low-level optimization.
- **Example**: Classic games and graphics engines leverage C for rendering and logic implementation.

5. Database Systems

- Large database systems like Oracle, MySQL, and PostgreSQL have components written in C due to its speed and memory control.

6. Networking and Communication

- C is used for network programming, including protocol implementation, socket programming, and server-client architectures.
- **Example**: Web servers, network routers, and switches require efficient network handling achievable through C.

7. Scientific Computing and Research

- High-performance numerical computation and simulations often rely on C for efficient algorithms and large-scale computation.
- **Example**: Weather modeling software, simulations, and computational physics programs use C for performance-critical calculations.

8. Education and Learning Programming Concepts

- C is widely used in computer science curricula to teach core concepts such as data structures, algorithms, memory management, and pointers.

- Learning C helps students understand how computers manage memory and execute instructions at a low level.

4.What is complied and interpreted programming languages

Complied:A complier can complied entire code at once and it is faster execution as compared to interpreter the compiler can be used in (c and c++)

Interpreted: An interpreted can executed line by line,it is slower,the interpreted is used in (Python,java script,ruby).

5. Compare C language over python or Java languages?

6. What is data type and explain about different data types supported in C?

- In C programming, a data type defines the type of data that a variable can hold. It specifies the size, storage, and range of values that a variable can store. Choosing the correct data type is essential for memory efficiency and accurate program operations.
- Data type describe the type of the data,(i,e.) it decide how much memory required and what is allocated to store in it.

```
                                   Data types
                                       |
    ——----------------------------------------------------------------------------------------
    |                        |                        |                        |
Primitive data type   Derived data type   User-defined data type   Empty data type
|-char                  |-Arrays              |-structure                 |-void
|-int                   |-string              |-union
|-float                 |-pointers            |-enum
|-double                |-function            |-typedef
```

7. What is difference between primitive data type and derived data type and user defined data type?

1. Primitive (Basic) Data Types: These are built in functions

Key Points:

- Represent fundamental values like numbers and characters.
- Typically used directly for computation or storage.
- Examples: `int`, `float`, `char`, `double`.

2. Derived Data Types::are created from primitive data types

Derived data types are types that are derived from primitive types to create more complex structures or collections of elements. They include:

- Arrays: collection of elements of same type.
- Pointers: variables storing memory addresses.
- Function types: return type or parameter type of functions.

3.User defined data type: user defined data types are created by the programar to store complex data

8. What is operator and explain about different types of operators?

Operators: In programming, an operator is a special symbol or keyword that tells the compiler or interpreter to perform specific operations on one or more operands (values or variables). Operators are fundamental building blocks in programming, used to create expressions and manipulate data.

1.**Arithmetic Operators:** +,-,*,/,%

2.**Relational operators:** Relational operators compare two values and return a Boolean value (`True` or `False`).

3.Logical operator:Logical operators are used to combine multiple conditions or Boolean expressions(&&,||,,^)

4.Bitwise operators: Bitwise operators perform operations on the binary representations of integers.(`&`,`>>`,`<<`,`^`,`|`)

5.Assignment operators:Assignment operators are used to assign values to variables

6.Conditional perators:(ternary operator)

7.Increment/Decrement operators:Increment (`++`) and decrement (`--`) operators are used to increase or decrease a numeric variable by 1. They come in two forms

8.Special operators: Special operators are used for specific purposes, varying by programming language. Common examples include:

- sizeof : Returns the size of a data type or variable in memory (`sizeof(int)`)
- & (address-of operator) : Returns the memory address of a variable in languages like C/C++
- * (pointer dereference) : Accesses the value at a pointer's address
- type casting operators : Convert a variable from one type to another (`(float)x`)
- comma operator : Separates multiple expressions where only one value is returned (`int x = (a++, b++)`)

9. What is precedence and associativity and write list of operators based on precedence?

Precedence:

Precedence defines the priority of operators in an expression.Operators with higher precedence are evaluated first

Associativity

Associativity determines the order in which operators of the **same precedence** are evaluated. It can be **left-to-right** or **right-to-left**

High precedence

( ),[ ],{ }	**&**
++,- -,!	**^**
***,/,%**	|
+,-	**&&**
<<,>>	||
<,>	**=**
==	**,**

10.What is printf() and scanf() functions and give example programs for it?

In C programming, `printf()` and `scanf()` are standard input/output functions provided by the `stdio.h` library.

1. **`printf()` Function:**Printf() is used to display a msg on the console window
2. **`scanf()` Function:** It can used to read an input from the user in output screen

11. What is statement and explain about different type of statements in C?

- In C programming, a **statement** is a single line of code that performs a specific task.

1.Simple statement: Each and every simple statement are independetn.The simple statements are end with semicolon**(;)**
2.Conditional statement: Conditional statements are also called as decision making statements and these statemetns must have keywords such as(if,else,else-if,switch)
3.Looping statement: Looping statements contains set of statements and these statements are executed as long as condition becomes true.(While.do_while,for)
4.Jumping statement:(break,continue,return)

12. Explain about conditional statements (if,else,else-if,switch)

1. if Statement

The if statement is used to execute a block of code when a condition is true.

2. else if Statement

The else if statement is used to check multiple conditions. If the first condition is false, the program checks the next condition**.**

3. else Statement

The else statement is used when all previous conditions are false. It executes a default block of code.

4. switch Statement

The switch statement is used to select one block of code from multiple options based on the value of a variable.

13. What is control statement and explain about while and for loop statements

Control Statement:

A control statement is used to control the flow of execution of a program. It determines which statements will execute, how many times they execute, and in what order.

Control statements help the program make decisions and repeat actions.

1. While Loop

The while loop is used to execute a block of code repeatedly as long as the condition is true.

```
#include <stdio.h>

int main()

{

    int i = 1;

    while(i <= 5)

    {

        printf("%d ", i);

        i++;

    }


   return 0;

}
```

2. For Loop

The for loop is used to repeat a block of code a specific number of times.

```
#include <stdio.h>

int main()
```

```
{
    int i;
    for(i = 1; i <= 5; i++)
    {
        printf("%d ", i);
    }
    return 0;
}
```

14. Explain differences between while and for loop with example program

The difference between while and for loop is , the while can we use if we dont know how many iterations are required

The for loop we can use the number of iterations are we required are known

```
#include <stdio.h>
int main()
{
    int i;
     // Using while loop
    i = 1;
    printf("Numbers using while loop:\n");
    while(i <= 5)
    {
        printf("%d ", i);
        i++;
    }
  // Using for loop
    printf("\nNumbers using for loop:\n");
    for(i = 1; i <= 5; i++)
    {
        printf("%d ", i);
    }
   return 0;
}
```

15. Explain differences between while and do-while loop with example programs

Feature	While Loop	Do-While Loop
Condition Check	Condition is checked before executing the loop	Condition is checked after executing the loop
Execution	May execute zero times if condition is false	Executes at least once even if condition is false
Syntax	`while(condition)`	`do { } while(condition);`
Use	When execution depends on the condition first	When the loop must run at least one time

WHILE:

```
#include <stdio.h>
int main()
{
   int i = 1;
   while(i <= 5)
   {
      printf("%d ", i);
      i++;
   }
   return 0;
}
```

DO WHILE:

```
#include <stdio.h>
int main()
{
   int i = 1;
   do
   {
      printf("%d ", i);
      i++;
   }
   while(i <= 5);
   return 0;}
```

16. What is a pointer and explain various advantages of pointers?

Pointer: A pointer is a variable that stores the memory address of another variable.Instead of storing the actual value, a pointer stores the location (address) of the variable in memory.

Advantages of Pointers

1. Dynamic Memory Allocation
   Pointers help in allocating memory dynamically using functions like `malloc()` and `calloc()`.
2. Efficient Array and String Handling
   Pointers make it easier and faster to access elements of arrays and strings.
3. Passing Arguments to Functions
   Pointers allow call by reference, which means a function can modify the original variable.
4. Memory Management
   Pointers help in managing and accessing memory efficiently.
5. Implementation of Data Structures
   Complex data structures like linked lists, stacks, queues, and trees are implemented using pointers.
6. Improves Program Efficiency
   Using pointers can reduce memory usage and improve performance.

17. Explain about Wild, NULL, Dangling and Generic Typed Pointers with example programs.

1. Wild Pointer

A wild pointer is a pointer that has not been initialized. It points to an unknown or random memory location, which may cause errors.

```
#include <stdio.h>
int main()
{
    int *p;   // wild pointer (not initialized)
   printf("%p", p);
    return 0;
}
```

2. NULL Pointer

A **NULL pointer** is a pointer that **does not point to any valid memory location**. It is assigned with the value **NULL**.

```
#include <stdio.h>
int main()
{
    int *p = NULL;
  if(p == NULL)
    {
        printf("Pointer is NULL");
    }
  return 0;
}
```

3. Dangling Pointer:

A **dangling pointer** is a pointer that **points to a memory location that has already been freed or deleted**.

```
#include <stdio.h>
#include <stdlib.h>
int main()
{
    int *p = (int*)malloc(sizeof(int));
   *p = 10;
    free(p);   // memory released
   printf("%d", *p);   // dangling pointer
  return 0;
}
```

4. Generic Pointer (Void Pointer)

A **generic pointer** is a pointer that **can store the address of any data type**. It is declared using `void`.

```
#include <stdio.h>
int main()
{
    int a = 10;
    void *p;
   p = &a;
   printf("%d", *(int*)p);  return 0; }
```

18. What is the difference between typed pointers and un typed pointers?

Difference Between Typed Pointers and Untyped Pointers

Pointers in C can be typed (associated with a specific data type) or untyped (can point to any data type). Here's the detailed comparison:

Feature	Typed Pointer	Untyped Pointer (Generic Pointer)
Definition	A pointer that points to a specific data type (int, float, char, etc.)	A pointer that can point to any data type (declared as `void *`)
Declaration	`int *p;` , `char *c;`	`void *p;`
Type Safety	Type safe — compiler knows what data type it points to	Not type safe — requires type casting before dereferencing
Dereferencing	Can be dereferenced directly	Must be type cast before dereferencing
Use Case	For normal pointer operations where data type is known	For generic functions or when pointer can point to any data type

19. What is Static memory allocation and Dynamic memory allocation and explain the differences between static and dynamic memory allocations.

1. Static Memory Allocation

Definition:
Static memory allocation is when memory is allocated at compile time. The size of the memory is fixed and cannot change during program execution.

2. Dynamic Memory Allocation

Definition:
Dynamic memory allocation is when memory is allocated at runtime as per program requirement. The size can be decided during execution.

Differences Between Static and Dynamic Memory Allocation

Feature	Static Memory Allocation	Dynamic Memory Allocation
Time of Allocation	Compile time	Run time
Flexibility	Fixed size, cannot change	Flexible size, can change during execution
Memory Wastage	May waste memory if size is overestimated	Efficient memory usage
Deallocation	Automatically done at the end of program	Must be manually freed using `free()`
Examples	`int a; int arr[10];`	`malloc()`, `calloc()`, `realloc()`

20. What is malloc(), calloc() functions with example programs and explain differences between malloc() and calloc() functions?

1.malloc:

Type conversion is mandatory before de-reference
.>Malloc()is used to create memory dynamically for single block only
.>it takes only one argument that is size of the specified data
.>it also return address of type void* on success and we must type cast the void*to specific type before de-reference

```
#include <stdio.h>
#include <stdlib.h>
int main() {
int *num, i, isPrime = 1;
num = (int*)malloc(sizeof(int));
for(i = 2; i <= *num / 2; i++) {
if(*num % i == 0) {
isPrime = 0;
break;
}
}
if(isPrime)
printf("%d is prime.\n", *num);
```

```
else
printf("%d is not prime.\n", *num);
free(num); // release memory
return 0;
}
```

2.Calloc

.>Calloc()function can be used to allocate the memory for multiple blocks contigeously
.>Calloc()function return first block (Base address)address to the user on success
.>Like malloc(),In calloc()function we must and should type cast in memory given by the calloc() before De-reference

```
#include <stdio.h>
int main()
{
   int n,*ptr,i;
  scanf("%d",&n);
   ptr=(int*)malloc(n,sizeof(int));
   for(i=0;i<i;i++){
       scanf("%d",ptr+i);
   }
   for(i=0;i<n;i++){
       printf("%d",*(ptr+i));
       }
 return 0;
}
```

Differences Between `malloc()` and `calloc()`

Feature	malloc()	calloc()
Memory Initialization	Not initialized (garbage values)	Initialized to 0
Parameters	1 parameter: size in bytes	2 parameters: number of elements and size of each element
Syntax	`malloc(size)`	`calloc(num, size)`

Use	Simple memory allocation	Allocate array memory and initialize
Speed	Slightly faster	Slightly slower because of initialization

21. What is realloc() function with example program and explain differences between realloc() calloc() functions

`realloc()` Function in C

The `realloc()` function is used to resize previously allocated memory. It can increase or decrease the size of a memory block allocated by `malloc()` or `calloc()` without losing the existing data.

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int *arr, i, n;

    printf("Enter initial number of elements: ");
    scanf("%d", &n);

    arr = (int*) malloc(n * sizeof(int));

    for(i = 0; i < n; i++) {
        arr[i] = i + 1;
    }

    printf("Original array: ");
    for(i = 0; i < n; i++)
        printf("%d ", arr[i]);
    printf("\n");

    // Resize array
    n = n + 3;
    arr = (int*) realloc(arr, n * sizeof(int));

    for(i = 3; i < n; i++)
        arr[i] = (i + 1) * 2;
```

```
    printf("Resized array: ");
    for(i = 0; i < n; i++)
        printf("%d ", arr[i]);
    printf("\n");

    free(arr);
    return 0;
}
```

Differences Between `realloc()` and `calloc()`

Feature	realloc()	calloc()
Purpose	Resizes previously allocated memory	Allocates new memory and initializes it to 0
Parameters	Pointer to old memory, new size	Number of elements, size of each element
Initialization	Does not initialize newly allocated memory	Initializes all memory to 0
Use Case	When you want to resize memory dynamically	When you want to allocate fresh memory for an array

22. Explain about free() function with example program?

`free()` Function in C

The `free()` function is used to release memory that was previously allocated dynamically using `malloc()`, `calloc()`, or `realloc()`.

- It prevents memory leaks by returning the allocated memory back to the system.
- After calling `free()`, the pointer becomes a dangling pointer if not set to NULL.

```
#include <stdio.h>

#include<stdlib.h>

int main()
```

```
{
   int *ptr;
   {
       int a=10;
       ptr=&a;
   }
   printf("%d\n",,*ptr);
   free(ptr);
   printf("%d",*ptr)
    return 0;
}
```

output:10

-164372(Garbage)

29. Differences between break and continue keywords with example programs

1. `break` Keyword

- Purpose: Immediately exits the loop or switch statement and transfers control to the next statement after the loop.

- Can be used in `for`, `while`, `do-while`, or `switch` statements.

```
#include <stdio.h>
int main() {
   int i;
  printf("Using break:\n");
    for(i = 1; i <= 10; i++) {
        if(i == 5) {
          break;  // exit loop when i is 5
```

```
        }
        printf("%d ", i);
    }
 return 0;
}
```

2. `continue` Keyword

- Purpose: Skips the current iteration of the loop and continues with the next iteration.
- Loop does not terminate, only skips the statements below `continue` in that iteration.

```
#include <stdio.h>
int main() {
    int i;
 printf("Using continue:\n");
    for(i = 1; i <= 10; i++) {
        if(i == 5) {
            continue;  // skip printing 5
        }
        printf("%d ", i);
    }
   return 0;
}
```

27. What enum data type and explain about enum data type with example program?

Enum Data Type in C

The `enum` **(enumeration) data type** in C is a **user-defined data type** that consists of a set of **named integer constants**.

- It helps to **make programs more readable** by using meaningful names instead of numbers.
- By default, the first name is assigned **0**, the next is **1**, and so on, unless explicitly assigned.

```
#include <stdio.h>

int main() {

    // Define enum for days of the week

    enum Week { Sunday, Monday, Tuesday, Wednesday, Thursday, Friday, Saturday
};

   enum Week today;

    today = Wednesday;

  printf("Value of today = %d\n", today);  // enums are stored as integers

    return 0;

}
```

26. What is structure and union and explain the differences between structure and union with example program?

Structure and Union in C

Both structure and union are user-defined data types in C that allow grouping different types of variables under a single name.

```
#include <stdio.h>

struct Student {

    int id;

    char name[20];

    float marks;};

int main() {
```

```
    struct Student s1 = {101, "Alice", 85.5};
  printf("ID: %d\n", s1.id);
    printf("Name: %s\n", s1.name);
    printf("Marks: %.2f\n", s1.marks);
   return 0;
}
```

2. Union (`union`)

- A union is similar to a structure but shares the same memory for all members.
- Only one member can store a value at a time.
- Total memory = size of the largest member.

```
#include <stdio.h>
union Data {
    int i;
    float f;
    char c;
};
int main() {
    union Data d;
    d.i = 10;
    printf("Integer: %d\n", d.i);
 d.f = 3.5;
    printf("Float: %.2f\n", d.f);
 d.c = 'A';
    printf("Char: %c\n", d.c);
  return 0;}
```

Differences Between Structure and Union

Feature	Structure	Union
Memory Allocation	Each member has separate memory	All members share the same memory
Access	Can access all members at the same time	Only one member can store value at a time
Size	Sum of sizes of all members	Size of largest member
Use Case	When you need all members simultaneously	When you need only one va

25. What is local and global variable and explain differences between local and global variables?

. Local Variable

- A local variable is declared inside a function or block.
- Scope: Accessible only within that function or block.
- Memory is allocated when the function is called and freed when the function ends.

2. Global Variable

- A global variable is declared outside all functions, usually at the top of the program.
- Scope: Accessible from any function in the program.
- Memory is allocated for the entire program execution.

Feature	Local Variable	Global Variable
Scope	Accessible only inside the function/block	Accessible throughout the program
Declaration	Inside function/block	Outside all functions
Lifetime	Exists only while function executes	Exists for entire program execution
Memory	Allocated on stack	Allocated in data segment
Default Value	Garbage (if not initialized)	0 (if not initialized)